

Agente Doom: percepción YOLO acoplada a un agente Double DQN

Sustentación del proyecto final. Un agente que juega Doom (ViZDoom) combinando un detector de objetos YOLOv8 como módulo de percepción con una política de aprendizaje por refuerzo Double DQN (Dueling + Prioritized Replay + n-step). Entrenado y evaluado en dos escenarios: el pasillo `deadly_corridor` y la sala abierta `defend_the_center`.

0.908

MAP@0.5 DEL DETECTOR
(PERCEPCIÓN, VALIDACIÓN)

Detector entrenado sobre un dataset in-domain auto-etiquetado desde el propio motor, sin anotación manual.

Equipo: Los Controladores

Estudiantes: Joshua Marín Castrillón (entorno + política) · David Henao Rojas (percepción)

Profesor: Diego Pérez Rosero

Fecha: Junio de 2026

SOLUCIÓN EN UNA LÍNEA

Cada frame del juego pasa por un **detector YOLOv8** que lo convierte en un vector semántico de 13 variables (hay enemigo, a qué lado, qué tan cerca, vida, munición). Tres frames se apilan en un estado de 39 dimensiones que alimenta una red **Double DQN con arquitectura Dueling**, entrenada con Prioritized Experience Replay y retornos n-step bajo una función de recompensa diseñada según el escenario.

39

DIMENSIONES DEL ESTADO (13 × 3 FRAMES)

13

ACCIONES DISCRETAS (MOVER, GIRAR, DISPARAR)

4

TÉCNICAS SOBRE EL DQN BASE (DOUBLE, DUELING, PER, N-STEP)

2

ESCENARIOS: PASILLO Y SALA ABIERTA

Jugar Doom es un problema de decisión secuencial bajo observación parcial

El agente no ve el mapa completo: solo el frame que tiene enfrente. En cada instante debe decidir una acción (girar, avanzar, disparar) para maximizar una recompensa a futuro (matar enemigos, sobrevivir, llegar a la meta). Esto es un **Proceso de Decisión de Markov parcialmente observable**. La dificultad de aprender directamente desde píxeles motiva la decisión central del diseño: **separar percepción de control**.

Por qué no aprender desde píxeles crudos

Un DQN sobre imágenes (240×320×3) debe aprender a la vez a "ver" y a "decidir". Eso exige millones de frames y una CNN profunda. Bajo cuota de GPU es inviable iterar.

Nuestra alternativa: percepción explícita

YOLO traduce el frame a un estado semántico de baja dimensión. El agente RL aprende solo el control sobre 39 números interpretables, no sobre 230.400 píxeles. Mucho más eficiente en muestras.

PIPELINE COMPLETO DE INFERENCIA (UN PASO DE JUEGO)



Percepción (David)

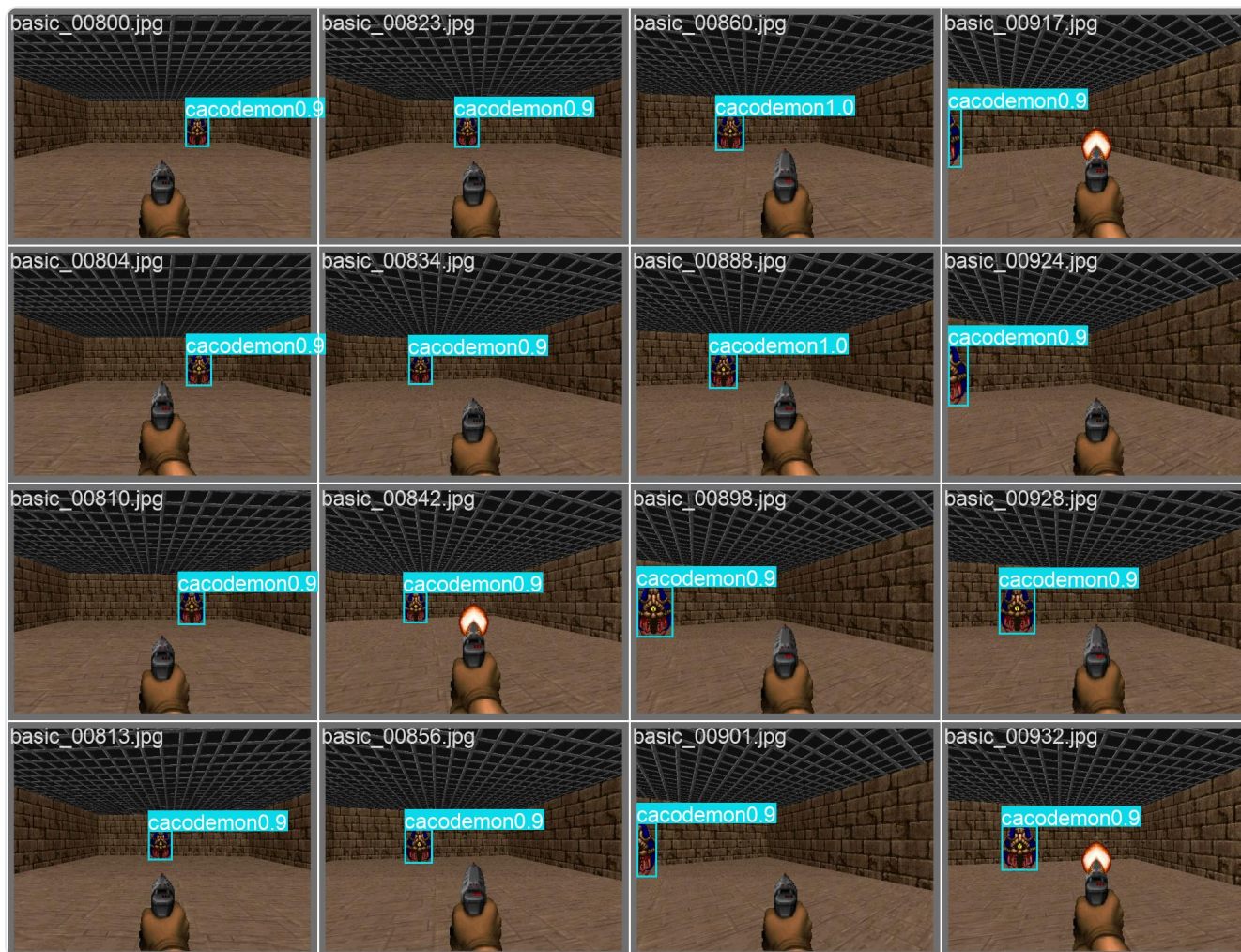
Detector YOLOv8s entrenado en dominio. Provee, por frame: clase y caja de cada enemigo, y vía el **depth buffer** de ViZDoom, la distancia. Es el "sistema visual" del agente.

Entorno y política (Joshua)

El wrapper RL construye el estado, define la recompensa según el escenario y entrena el agente Double DQN. Es el "sistema de decisión": qué hacer con lo que se ve.

De píxeles a un estado semántico de 13 variables

El detector no se entrenó con datos genéricos de internet sino con imágenes del **propio motor del juego**, lo que elimina la brecha de dominio que hacía "alucinar" enemigos en las paredes. La clave: ViZDoom expone un `labels_buffer` con la posición y el nombre exactos de cada objeto en pantalla, lo que permite generar las cajas ground-truth automáticamente.



Detector YOLOv8 sobre frames de validación: cada caja es un enemigo localizado con su clase y confianza (aquí, *cacodemon*). Estas cajas son la materia prima del vector de estado.

1

Captura + auto-etiquetado desde el motor

Se recorren cuatro escenarios (`basic`, `deadly_corridor`, `defend_the_center`, `freedom`) con acciones aleatorias. Por cada frame, el `labels_buffer` entrega las cajas reales. Se conserva un 20% de frames vacíos como fondo para enseñar a no detectar nada cuando no hay nada.

2

Entrenamiento del detector (YOLOv8s, 100 épocas, 640 px)

2.213 imágenes etiquetadas (1.817 entrenamiento, 396 validación). El resultado es el modelo `doom-v4`, que se versiona en el repositorio para reproducir el pipeline sin re-etiquetar.

3

Proyección a 13 features normalizadas

Las cajas más vida y munición se comprimen en el vector de estado. Solo se confía en las 4 clases que el detector reconoce de forma fiable: `zombieman`, `imp`, `pinky`, `shotgun-guy`.

EL VECTOR DE ESTADO (13 DIMENSIONES)

■ 0–3 presencia de enemigo, offset horizontal del más

■ 8 pasos sin ver enemigo (urgencia de explorar).

Un detector fiable es condición necesaria para la política

Si la percepción alucina, el agente aprende a reaccionar a fantasmas. Por eso se validó el detector antes de entrenar la política, y se restringió la recompensa a las clases realmente confiables.

0.930

PRECISIÓN
(VALIDACIÓN)

0.783

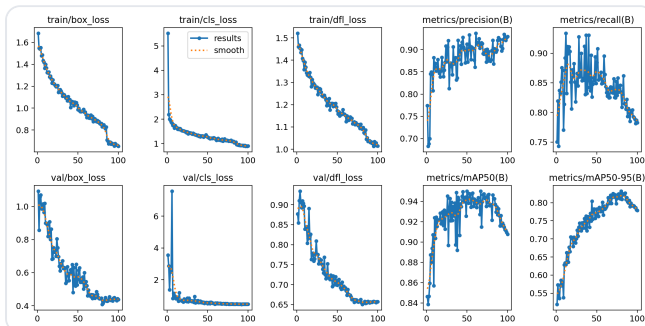
RECALL
(VALIDACIÓN)

0.908

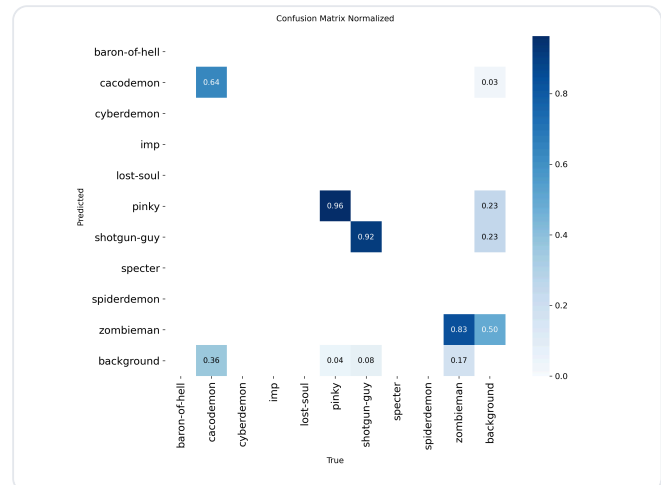
MAP@0.5

0.779

MAP@0.5:0.95



Curvas de entrenamiento (100 épocas): las pérdidas de caja, clase y DFL bajan de forma estable y el mAP se asienta sobre 0.90 sin sobreajuste evidente.



Matriz de confusión normalizada. Diagonal fuerte en las clases usadas: pinky 0.96, shotgun-guy 0.92, zombieman 0.83.

LECTURA HONESTA DE LA MATRIZ: POR QUÉ SOLO 4 CLASES

La matriz revela el límite real del detector y justifica una decisión de diseño:

- **Clases fiables:** pinky, shotgun-guy y zombieman se reconocen con acierto alto (0.83–0.96). Son las que aparecen en los escenarios jugados y las únicas que cuentan para la recompensa.
- **Clases descartadas:** cacodemon se confunde con fondo un 36% del tiempo, y varias clases (baron-of-hell, cyberdemon, imp, lost-soul, spiderdemon) no tienen muestras de validación suficientes. Incluirlas solo sumaría falsos positivos.
- **Salvaguarda en la recompensa:** aun con un detector bueno, para que una detección "cuenta" como enemigo de combate se exige una confianza de **0.50**, más estricta que el umbral del detector (0.40). Es el dial que evita que el agente cobre recompensa por dispararle a un falso positivo.

La decisión de confiar solo en cuatro clases no es una limitación oculta: es el resultado de leer la matriz de confusión y preferir una percepción estrecha pero fiable a una amplia pero ruidosa.

El corazón del agente: aproximar la función Q óptima

El objetivo del agente es encontrar una política que maximice el retorno descontado. El valor de tomar la acción a en el estado s y luego actuar de forma óptima se llama función de acción-valor Q^* , y satisface la ecuación de Bellman:

$$Q^*(s,a) = \mathbb{E}_s[r + \gamma \max_{a'} Q^*(s',a')]$$

Bellman óptima

Con factor de descuento $\gamma = 0.99$: el agente valora el futuro casi tanto como el presente, necesario porque las recompensas (kills, llegar a la meta) llegan con retraso respecto a las acciones que las causaron.

1. APROXIMACIÓN CON RED NEURONAL Y OBJETIVO DOUBLE DQN

Como el espacio de estados es continuo, se aproxima Q con una red de parámetros θ . El DQN estándar sobreestima los valores porque usa el mismo max para elegir y evaluar. **Double DQN** lo corrige: una red (θ) **elige** la mejor acción y la red objetivo (θ^-) la **evalúa**.

$$y = \sum_{k=0}^{N-1} \gamma^k r_{t+k} + \gamma^N Q(s_{t+N}, \operatorname{argmax}_{a'} Q(s_{t+N}, a'; \theta); \theta^-) \cdot (1 - d)$$

objetivo n-step Double DQN

El primer término acumula las recompensas reales de $N = 3$ pasos (retorno n-step): da crédito más preciso a secuencias de acciones, en vez de propagar de a un paso. El segundo término es el "bootstrap": el valor estimado del estado a N pasos, descontado por γ^N . El factor $(1 - d)$ anula el bootstrap si el episodio terminó ($d = 1$).

2. ARQUITECTURA DUELING

La red no estima Q directamente, sino que la descompone en el **valor del estado** $V(s)$ (qué tan bueno es estar aquí) y la **ventaja** $A(s,a)$ (cuánto mejor es cada acción frente al promedio):

$$Q(s,a) = V(s) + (A(s,a) - (1/|\mathcal{A}|) \sum_{a'} A(s,a'))$$

Dueling

Restar la media de la ventaja resuelve la ambigüedad de identificabilidad (sin ella, sumar una constante a V y restarla a A daría la misma Q). La ganancia práctica: el agente actualiza $V(s)$ en cada paso aunque no haya probado todas las acciones, así aprende más rápido qué estados son peligrosos.

3. EL ERROR QUE SE MINIMIZA

El aprendizaje empuja la predicción hacia el objetivo. El residuo es el error de diferencia temporal (TD):

$$\delta_t = y - Q(s_t, a_t; \theta)$$

TD-error

Este δ cumple doble papel: define la pérdida (página siguiente) y, en valor absoluto, mide qué tan "sorprendente" fue una transición, lo que el Prioritized Replay aprovecha para muestrear.

Cómo aprende, de qué muestras y cómo explora

4. PRIORITIZED EXPERIENCE REPLAY (PER)

En vez de muestrear el buffer de forma uniforme, se priorizan las transiciones con mayor TD-error (de las que más hay que aprender). La prioridad y la probabilidad de muestreo son:

$$p_i = |\delta_i| + \varepsilon, \quad P(i) = p_i^\alpha / \sum_k p_k^\alpha$$

prioridad y muestreo

Con $\alpha = 0.6$: interpola entre muestreo uniforme ($\alpha=0$) y puramente proporcional a la prioridad ($\alpha=1$). El muestreo proporcional es $O(\log N)$ gracias a un árbol de sumas (*SumTree*).

Priorizar introduce un sesgo: las transiciones "difíciles" aparecen más de lo que deberían. Se corrige con pesos de muestreo por importancia (IS):

$$w_i = (1 / (N \cdot P(i)))^\beta / \max_j w_j$$

corrección de sesgo (IS)

El exponente β sube de 0.4 a 1.0 a lo largo del entrenamiento (annealing): la corrección importa más al final, cuando la política ya casi convergió. Se normaliza por el máximo para que los pesos solo reduzcan, nunca amplifiquen, el tamaño del paso.

5. FUNCIÓN DE PÉRDIDA

La pérdida es el error de Huber (suave: cuadrático cerca de cero, lineal lejos, robusto a outliers), ponderado por los pesos IS y promediado en el lote de tamaño $B = 512$:

$$L(\theta) = (1/B) \sum_i w_i \cdot \text{Huber}(\delta_i)$$

pérdida ponderada

Optimizada con Adam ($lr = 5 \times 10^{-4}$ con recocido coseno) y recorte de gradiente a norma 10. La red objetivo θ^- se sincroniza con θ cada 2.000 pasos, lo que estabiliza el objetivo móvil.

6. EXPLORACIÓN: E-GREEDY DECRECIENTE

El agente explora al azar con probabilidad ε y explota su política en otro caso. ε decae linealmente de exploración total a casi pura explotación:

$$\varepsilon(t) = \varepsilon_0 + \min(1, t/T) \cdot (\varepsilon_f - \varepsilon_0) \quad [1.0 \rightarrow 0.05 \text{ en } T = 50.000 \text{ pasos}]$$

ε -greedy

Además, en la sala se enmascaran las acciones de avanzar: sus valores Q se fijan a $-\infty$ antes del argmax, forzando un comportamiento de torreta sin necesidad de reentrenar la red.

Resumen del flujo de un paso de aprendizaje

Observar estado $\rightarrow$ elegir acción (ε -greedy con máscara) $\rightarrow$ ejecutar y observar (r, s') $\rightarrow$ acumular retorno n-step $\rightarrow$ guardar en el buffer priorizado $\rightarrow$ muestrear lote según prioridad $\rightarrow$ calcular objetivo Double DQN $\rightarrow$ minimizar Huber ponderado por IS $\rightarrow$ actualizar prioridades con el nuevo $|\delta|$.

La recompensa codifica qué significa "jugar bien" en cada sala

El reward nativo de ViZDoom premia la velocidad de avance, lo que el agente aprende a explotar (farmear) sin completar el nivel. Se descarta y se construye una recompensa propia, distinta según la geometría del escenario.



deadly_corridor : enemigos flanqueando un pasillo. Objetivo: avanzar hasta el chaleco del fondo sin morir.



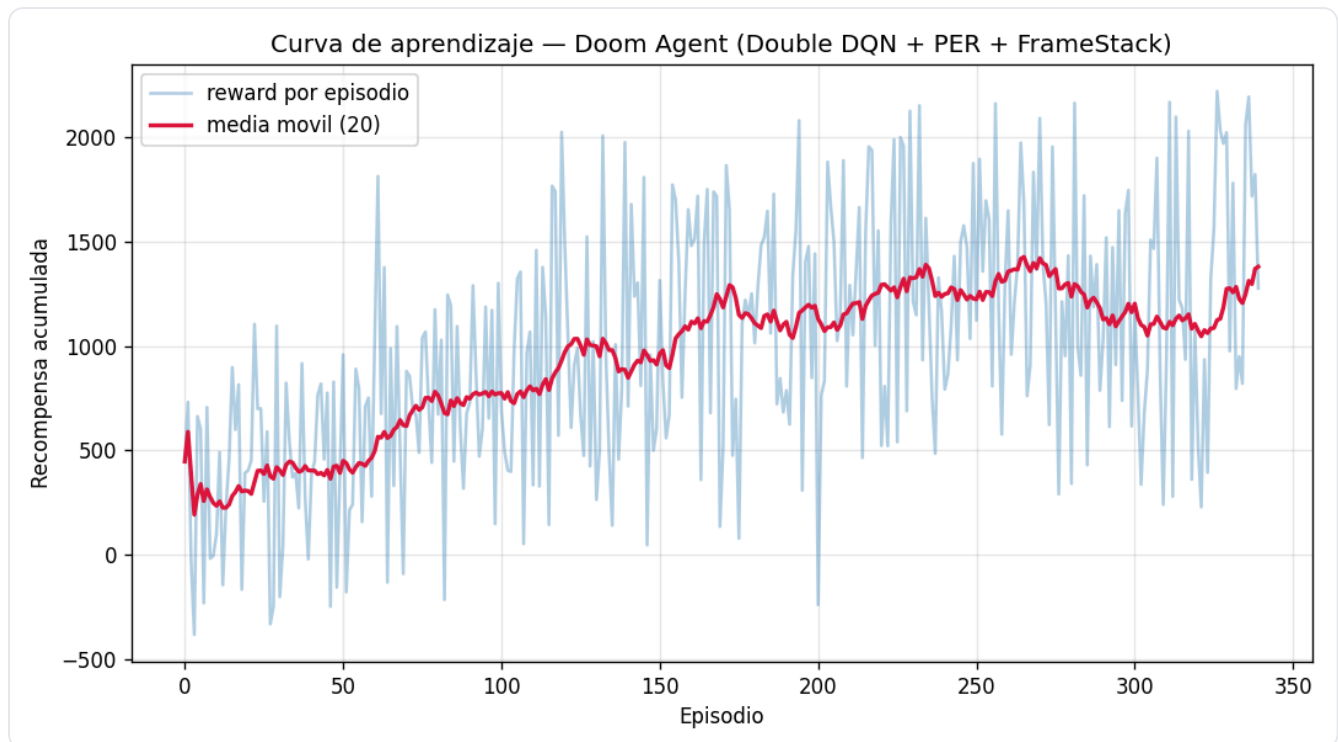
defend_the_center : arena con el jugador en el centro. Objetivo: sobrevivir girando y disparando a lo que aparece.

JERARQUÍA DE LA RECOMPENSA (DE MAYOR A MENOR MAGNITUD)

COMPONENTE	VALOR	RAZÓN DE DISEÑO
Completar nivel (pasillo)	+300	Premio terminal que domina cualquier farmeo: llegar vivo al chaleco antes del timeout.
Matar enemigo	+150	Objetivo principal. Señal fuerte y dispersa, una por baja confirmada.
Morir	-100	Castigo explícito; sin él, "lanzarse y morir" salía rentable.
Sobrevivir la ronda (sala)	+50	Equivalente de la meta en la sala: aguantar hasta el timeout.
Disparar a la nada	-5	Penaliza malgastar munición; rompe el "rociar paredes".
Disparar con blanco	+2	Señal densa que guía hacia el comportamiento de combate.
Esquivar (strafe) con enemigo	+0.8	Fomenta evasión lateral cuando hay amenaza.
Daño recibido	$-0.8 \cdot \Delta$	Proporcional a la vida perdida: enseña a cubrirse.
Progreso (pasillo)	$+0.2 \cdot \Delta x$	Solo por terreno nuevo ganado hacia la meta (delta de la X máxima). No es velocidad, así que ir y venir no farmea.
Encarar / barrer (sala)	+0.2 / +0.05	Premia centrar al enemigo y girar para escanear cuando no se ve ninguno.
Costo por paso (pasillo)	-0.05	Desincentiva merodear; en la sala se sustituye por un bono de supervivencia.

La idea central del progreso "no farmeable": recompensar solo el avance monótono hacia la meta (cada unidad de terreno nuevo, una sola vez) en lugar de la velocidad. Así el óptimo de la

El agente aprende: la recompensa media crece de forma sostenida



Curva de aprendizaje en `defend_the_center`. La media móvil (20 episodios, en rojo) sube de ~300 a ~1.400 de recompensa acumulada: evidencia de que la política mejora con la experiencia, no de ruido. La corrida extendida en Kaggle continúa este entrenamiento hasta 1.000.000 de pasos.

~4.7×

CRECIMIENTO DE LA RECOMPENSA
MEDIA (300 → 1.400)

39 → 13 → 1

ESTADO → TRONCO → CABEZAS
V Y A DE LA RED DUELING

1M

PASOS DE LA CORRIDA
FINAL EN KAGGLE (GPU)

CUATRO CONCLUSIONES

- 1 **Separar percepción de control fue la decisión acertada.** Aprender sobre 39 features semánticas en vez de píxeles crudos hizo el problema tratable bajo cuota de GPU, y dejó una política interpretable.
- 2 **El auto-etiquetado in-domain resolvió las alucinaciones.** Entrenar el detector con frames del propio motor (mAP 0.908) eliminó la brecha de dominio; leer la matriz de confusión guió la decisión de confiar solo en cuatro clases.
- 3 **El diseño de la recompensa importa tanto como el algoritmo.** Descartar el reward nativo farmeable y construir un progreso no farmeable más eventos jerárquicos fue lo que alineó el incentivo del agente con completar el nivel.
- 4 **Las extensiones del DQN se justifican una a una.** Double corrige sobreestimación, Dueling acelera el aprendizaje de valor, PER enfoca las muestras difíciles y n-step mejora la asignación de crédito. Cada una responde a un problema concreto del DQN base.

TRABAJO FUTURO Y ENTREGA

Líneas de mejora: ampliar el dataset a más clases de enemigos, comparar contra un baseline de DQN sobre píxeles para cuantificar la ganancia de la percepción explícita, y extender el presupuesto de pasos. La solución completa es un notebook autocontenido (`doom_entrega.ipynb`) que muestra el código de cada módulo inline, carga el detector ya entrenado desde el repositorio y entrena la política en GPU.